

# コンピュータビジョン

担当: 井尻 敬

## Contents

01. 序論 : イントロダクション
02. 特徴検出1 : テンプレートマッチング、コーナー検出、エッジ検出
03. 特徴検出2 : ハフ変換、DoG, SIFT特徴
04. 領域分割 : 領域分割とは、閾値法、領域拡張法、グラフカット法、
05. オプティカルフロー : 領域分割残り, Lucas-Kanade法
06. パターン認識基礎1 : パターン認識概論, サポートベクタマシン
07. パターン認識基礎2 : ニューラルネットワーク、深層学習
08. パターン認識基礎3 : 主成分分析, オートエンコーダ
09. 筆記試験
10. プログラミング演習 1: PC室
11. プログラミング演習 2: PC室
12. プログラミング演習 3: PC室
13. プログラミング演習 4: PC室
14. プログラミング演習 5: PC室

## Contents :

オプティカルフロー

画像領域分割の続き

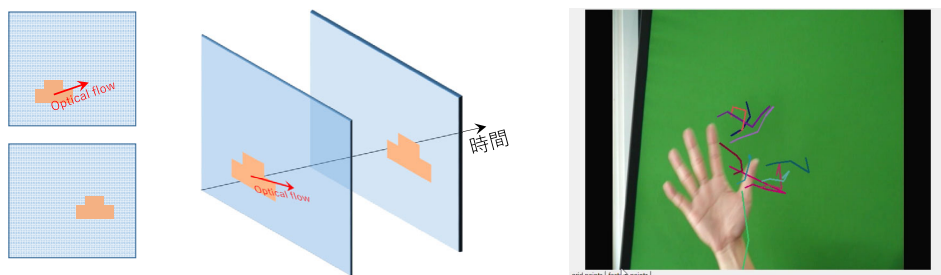
- 動的輪郭モデル
- 曲面再構成法
- 深層学習

モーフォロジー演算

オプティカルフロー

## オプティカルフロー

- 2枚の画像内におけるある物体の移動量をベクトルとして表現したもの
- 動画内の『物体追跡』や『物体の動きの解析』などに利用される
- ブロックマッチング法, Lucas-Kanade法など



5

## ブロックマッチング法

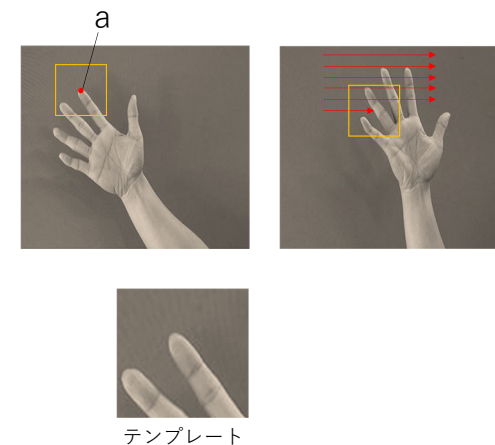
テンプレートマッチングにより  
オプティカルフローを求める手法

**入力:** 2枚の画像A・B、画像A内の点a

**出力:** 点aの移動先

**手法:**

- 点aを中心とする窓領域を作成
  - 窓領域をテンプレートとし、画像Bに対し、テンプレートマッチングを実施
  - SSDが最小となる場所を移動先として決定する
- ※ 移動距離が大きくないと仮定できる場合は、点aの周囲のみでテンプレートマッチングを実施する



追跡したい点ごとにテンプレートマッチング  
を実施するため計算量が大きくなる

6

## 復習: 連立一次方程式の行列を利用した表現

2変数の連立1次方程式を考える

$$\begin{aligned} 1x + 2y &= 1 \\ 3x + 4y &= 2 \end{aligned}$$



行列を使って以下の通り書き表せる

$$\begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

両辺に逆行列をかけることで解が得られる



$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}^{-1} \begin{pmatrix} 1 \\ 2 \end{pmatrix}$$

※ 逆行列が存在しない場合、この  
連立方程式は不定or不能

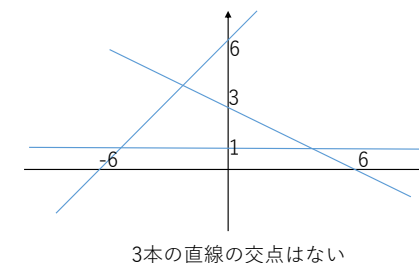
7

## 復習: 一般逆行列 (擬似逆行列)

2変数の連立1次方程式を考える

$$\begin{aligned} 1x + 2y &= 6 \\ -1x + 1y &= 6 \\ 0x + 1y &= 1 \end{aligned}$$

変数が2個、方程式が3本なので解なし



3本の直線の交点はない

8

## 復習：一般逆行列（擬似逆行列）

$$\begin{aligned} 1x + 2y &= 6 \\ -1x + 1y &= 6 \\ 0x + 1y &= 1 \end{aligned}$$

$$\rightarrow \begin{pmatrix} 1 & 2 \\ -1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 6 \\ 6 \\ 1 \end{pmatrix} \quad \text{行列を使って書き表した}$$

$$\rightarrow \mathbf{A} \begin{pmatrix} x \\ y \end{pmatrix} = \mathbf{b}, \quad \text{ただし } \mathbf{A} = \begin{pmatrix} 1 & 2 \\ -1 & 1 \\ 0 & 1 \end{pmatrix}, \quad \mathbf{b} = \begin{pmatrix} 6 \\ 6 \\ 1 \end{pmatrix} \quad \text{として整理した}$$

$$\rightarrow \mathbf{A}^T \mathbf{A} \begin{pmatrix} x \\ y \end{pmatrix} = \mathbf{A}^T \mathbf{b} \quad \text{両辺に } \mathbf{A}^T \text{ をかけた}$$

$$\rightarrow \begin{pmatrix} x \\ y \end{pmatrix} = (\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T \mathbf{b} \quad \text{両辺に } (\mathbf{A}^T \mathbf{A})^{-1} \text{ をかけた}$$

この  $(\mathbf{A}^T \mathbf{A})^{-1} \mathbf{A}^T$  を一般逆行列とよび、  
今回の例ではすべての方程式を“なるべく”満たす解が得られます

かなり雑な説明です。詳しくは線形代数の教科書等を参考にしてください。

9

$$\begin{aligned} 1x + 2y &= 6 \\ -1x + 1y &= 6 \\ 0x + 1y &= 1 \end{aligned}$$

$$\begin{pmatrix} 1 & 2 \\ -1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 6 \\ 6 \\ 1 \end{pmatrix}$$

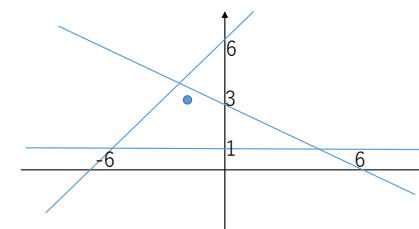
$$\begin{pmatrix} 1 & -1 & 0 \\ 2 & 1 & 1 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ -1 & 1 \\ 0 & 1 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 & -1 & 0 \\ 2 & 1 & 1 \end{pmatrix} \begin{pmatrix} 6 \\ 6 \\ 1 \end{pmatrix}$$

$$\begin{pmatrix} 2 & 1 \\ 1 & 6 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 0 \\ 19 \end{pmatrix}$$

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 6 \end{pmatrix}^{-1} \begin{pmatrix} 0 \\ 19 \end{pmatrix}$$

$$\begin{pmatrix} x \\ y \end{pmatrix} = \frac{1}{11} \begin{pmatrix} -19 \\ 38 \end{pmatrix}$$

## 復習：一般逆行列（擬似逆行列）



10

$$\begin{aligned} 1x + 2y &= 6 \\ -1x + 1y &= 6 \\ 0x + 5y &= 5 \end{aligned}$$

$$\begin{pmatrix} 1 & 2 \\ -1 & 1 \\ 0 & 5 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 6 \\ 6 \\ 5 \end{pmatrix}$$

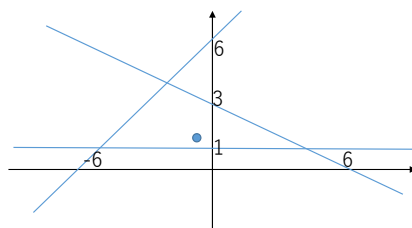
$$\begin{pmatrix} 1 & -1 & 0 \\ 2 & 1 & 5 \end{pmatrix} \begin{pmatrix} 1 & 2 \\ -1 & 1 \\ 0 & 5 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 1 & -1 & 0 \\ 2 & 1 & 5 \end{pmatrix} \begin{pmatrix} 6 \\ 6 \\ 5 \end{pmatrix}$$

$$\begin{pmatrix} 2 & 1 \\ 1 & 30 \end{pmatrix} \begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 0 \\ 43 \end{pmatrix}$$

$$\begin{pmatrix} x \\ y \end{pmatrix} = \begin{pmatrix} 2 & 1 \\ 1 & 30 \end{pmatrix}^{-1} \begin{pmatrix} 0 \\ 43 \end{pmatrix}$$

$$\begin{pmatrix} x \\ y \end{pmatrix} = \frac{1}{59} \begin{pmatrix} -43 \\ 86 \end{pmatrix} = \begin{pmatrix} -0.729 \\ 1.457 \end{pmatrix}$$

## 復習：一般逆行列（擬似逆行列）

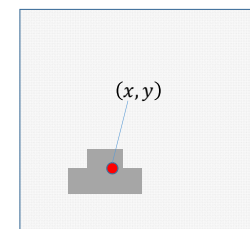


11

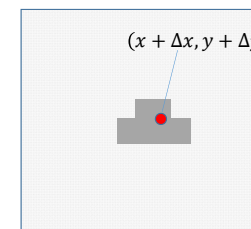
## Lucas-Kanade法 0

時刻  $t$  における画像、画像内の点  $(x, y)$ 、  
および 時刻  $t + \Delta t$  における画像が与えられる

→ このもとで点  $(x, y)$  の移動先  $(x + \Delta x, y + \Delta y)$  を求めたい



時間  $t$



時間  $t + \Delta t$

12

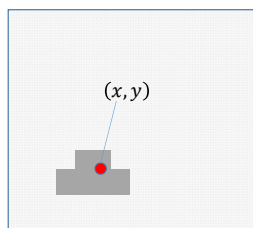
# Lucas-Kanade法 1

- 仮定1. 追跡対象の物体は移動後も同じ輝度値を持つ
- 仮定2. 移動量は非常に小さい

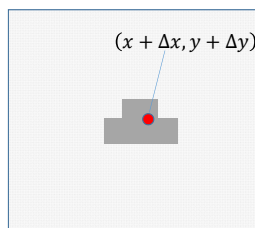
時刻  $t$  における点  $(x, y)$  が、時刻  $t + \Delta t$  で  $(x + \Delta x, y + \Delta y)$  に移動したとすると以下が成り立つ

$$I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t)$$

$I(x, y, t)$  は時刻  $t$  における画素  $(x, y)$  の輝度値を表す



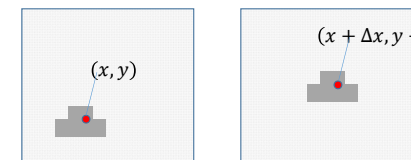
時間  $t$



時間  $t + \Delta t$

13

# Lucas-Kanade法 2



$$I(x, y, t) = I(x + \Delta x, y + \Delta y, t + \Delta t)$$

$$I(x, y, t) \approx I(x, y, t) + \frac{\partial I(x, y, t)}{\partial x} \Delta x + \frac{\partial I(x, y, t)}{\partial y} \Delta y + \frac{\partial I(x, y, t)}{\partial t} \Delta t$$

右辺をテーラー展開  
2次以上の項を無視

$$I_x \Delta x + I_y \Delta y + I_t \Delta t = 0$$

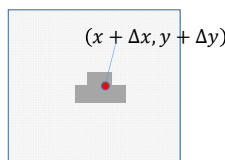
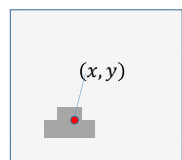
画像の  $x, y$  方向微分を  $I_x, I_y$  と、  
時間方向微分を  $I_t$  と表記

$$I_x u + I_y v + I_t = 0$$

両辺を  $\Delta t$  で除算し、 $\frac{\Delta x}{\Delta t} = u, \frac{\Delta y}{\Delta t} = v$  とおいた

# Lucas-Kanade法 3

$$I_x u + I_y v + I_t = 0$$



- この式はオプティカルフローの拘束式と呼ばれる

$I_x = \frac{\partial I(x, y, t)}{\partial x}$  は、画像  $I$  の  $(x, y)$  における横方向微分値 (Sobelフィルタでもとまる)

$I_y = \frac{\partial I(x, y, t)}{\partial y}$  は、画像  $I$  の  $(x, y)$  における縦方向微分値 (Sobelフィルタでもとまる)

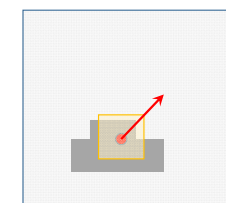
$I_t = \frac{\partial I(x, y, t)}{\partial t}$  画像  $I$  の  $(x, y)$  における時間方向微分値、 $(I(x, y, t + \Delta t) - I(x, y, t)) / \Delta t$  で計算できる

- 求めたいのは、追跡したい画素  $(x, y)$  の速度ベクトル  $(u, v)$
- 未知変数2個、方程式1本なのでこの問題は解けない (不定)

# Lucas-Kanade法 4

**仮定3** : 近傍画素は似た速度ベクトルを持つ

- 物体は複数画素から構成されるため



注目点  $(x, y)$  を中心とする窓領域内の画素  $(x_1, y_1), (x_2, y_2), \dots, (x_n, y_n)$  について、すべてが同じ速度ベクトル  $(u, v)$  を持つとすると...

$$I_x(x_1, y_1, t)u + I_y(x_1, y_1, t)v = -I_t(x_1, y_1, t)$$

$$I_x(x_2, y_2, t)u + I_y(x_2, y_2, t)v = -I_t(x_2, y_2, t)$$

:

$$I_x(x_n, y_n, t)u + I_y(x_n, y_n, t)v = -I_t(x_n, y_n, t)$$

が成り立つ

## Lucas-Kanade法 5

$$\begin{aligned} I_x(x_1, y_1, t)u + I_y(x_1, y_1, t)v &= -I_t(x_1, y_1, t) \\ I_x(x_2, y_2, t)u + I_y(x_2, y_2, t)v &= -I_t(x_2, y_2, t) \\ &\vdots \\ I_x(x_n, y_n, t)u + I_y(x_n, y_n, t)v &= -I_t(x_n, y_n, t) \end{aligned}$$

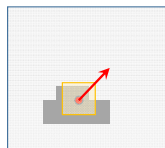
$$\begin{pmatrix} I_x(x_1, y_1, t) & I_y(x_1, y_1, t) \\ I_x(x_2, y_2, t) & I_y(x_2, y_2, t) \\ \vdots & \vdots \\ I_x(x_n, y_n, t) & I_y(x_n, y_n, t) \end{pmatrix} \begin{pmatrix} u \\ v \end{pmatrix} = - \begin{pmatrix} I_t(x_1, y_1, t) \\ I_t(x_2, y_2, t) \\ \vdots \\ I_t(x_n, y_n, t) \end{pmatrix} \dots (1)$$

この連立方程式は、変数2個、制約式n本なので解なし（不能）  
→ 一般逆行列により、なるべく全ての方程式を満たす解が得られる

Lucas-Kanade法の利点は高速であること

必要な計算は

- ・ 着目点周囲のx/y/時間方向微分
- ・ 一般逆行列の計算のみ



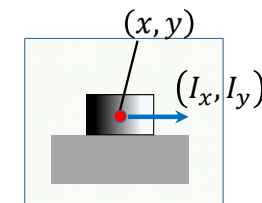
## オプティカルフローの拘束式の定性的な理解

$$-(I_x u + I_y v) = I_t$$

$(I_x, I_y)$  : 点(x,y)における輝度勾配

$(u, v)$  : 点(x,y)にある物体の速度

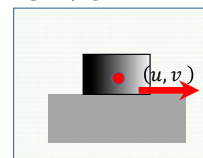
$I_t$  : 点(x,y)の輝度の時間変化



注目点にグラデーションがある状況を考える

左の例では勾配は右向き

速度が右向の状況を考えてみる...



左辺:  $-(I_x u + I_y v)$

- 輝度勾配と速度が同じ向き → 括弧の中身は正、全体は負
- 右図を少し右に動かすと注目点は暗くなる

左辺:  $I_t$

- 注目点(x,y)における実際の輝度変化
- もし右に進んでいるなら暗くなるので負になる

18

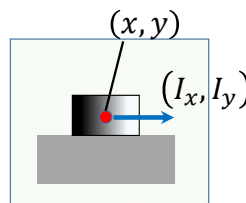
## オプティカルフローの拘束式の定性的な理解

$$-(I_x u + I_y v) = I_t$$

$(I_x, I_y)$  : 点(x,y)における輝度勾配

$(u, v)$  : 点(x,y)にある物体の速度

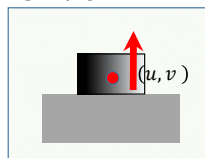
$I_t$  : 点(x,y)の輝度の時間変化



注目点にグラデーションがある状況を考える

左の例では勾配は右向き

速度が上向の状況を考えてみる...



左辺:  $-(I_x u + I_y v)$

- 輝度勾配と速度が直角 → 括弧の中身はゼロ、全体はゼロ
- 右図を少し上に動かしても注目点の色は変化なし

左辺:  $I_t$

- 注目点(x,y)における実際の輝度変化
- もし上に進んでいるなら色は変化しないので  $I_t = 0$

19

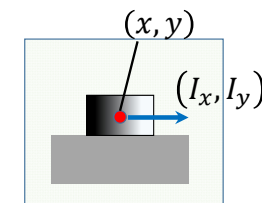
## オプティカルフローの拘束式の定性的な理解

$$-(I_x u + I_y v) = I_t$$

$(I_x, I_y)$  : 点(x,y)における輝度勾配

$(u, v)$  : 点(x,y)にある物体の速度

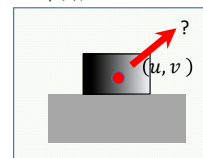
$I_t$  : 点(x,y)の輝度の時間変化



注目点にグラデーションがある状況を考える

左の例では勾配は右向き

本来は速度ベクトルが未知



この拘束式は、、、

注目点における「輝度勾配」と「輝度の時間変化」が既知の下で、それを満たす移動速度 (u, v) が満たすべき式を表す

左図にて  $I_t = -1$  なら、どっちにどの程度動く？

左図にて  $I_t = 0$  なら、どっちにどの程度動く？

左図にて  $I_t = +1$  なら、どっちにどの程度動く？

20

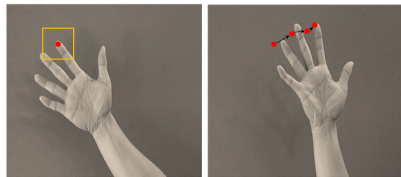
# Lucas-Kanade法 6

ここまでの説明したものは実際の動画に対してはあまりうまく動かない  
これは『仮定2: 移動量は非常に小さい』が実世界データでは成り立たないため  
※ 移動量は1画素程度であって欲しいが、追跡対象が1フレーム間に数画素分動いてしまうことも多い

## 1) 逐次更新

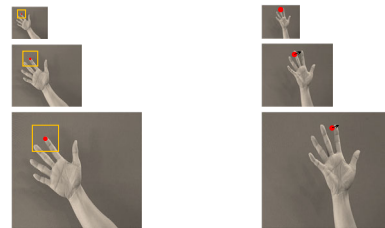
先の一般逆行列による解法を複数回繰り返すことで、移動ベクトルを更新

更新時の移動ベクトルにより、前頁(1)の右辺を計算する



## 2) 画像ピラミッドを利用した拡張

対象画像に対しガウシアンピラミッドを作成  
高い層で移動ベクトルを取得し、次の層ではそのベクトルを初期解として、逐次更新を実施



# RAFTによるOptical flow計算

現在はOptical Flow計算にも便利なNNモデルが利用可能 → RAFTを動かしてみる

## Step0 仮想環境を作ってactivate (飛ばしてもOK)

フォルダを作成しそこをcurrentにするcommand promptで

```
> python -m venv cv_sample
> .\cv_sample\Scripts\activate.bat
```

## Step1 必要なライブラリをインストール

Cuda + torchを使う場合は <https://pytorch.org/> からインストールコマンドを取得してインストール

```
> pip install torch torchvision torchaudio --index-url
https://download.pytorch.org/whl/cu118
```

CPUでよければ以下でもOK

```
> pip install torch torchvision numpy opencv-python
```

## Step2 右のコードを実行

```
> python rafttest.py
```

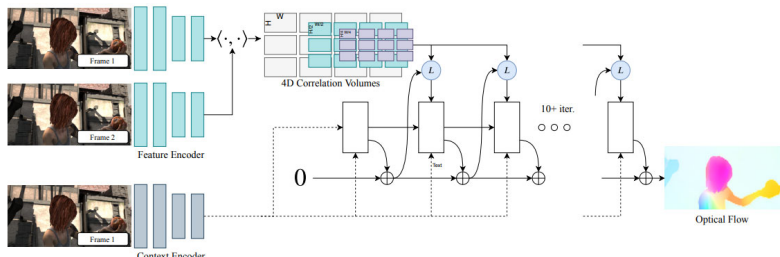


# RAFTによるOptical flow計算

補足資料  
Github : <https://github.com/princeton-vl/RAFT>  
論文 : <https://arxiv.org/abs/2003.12039>

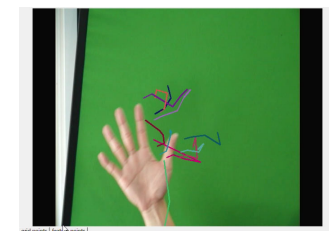
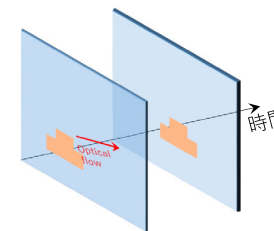
RAFT: Recurrent All-Pairs Field Transforms for Optical Flow

- 画像1 画像2についてfeature encoderで特徴マップを生成 (画像1にはContext encoderも適用)
  - Feature encoderは画像1と画像2に対して同じものを利用
- 画像1feature mapの全画素 対 画像2の全画素の内積をとりvolumeを生成
  - Feature mapが  $w \times h$  なら、 $w \times h \times w \times h$  の volume が現れる (これが画素間の類似度を表す & ピラミッド化する)
- 最後にRecurrent層 (図中の白い箱 GRU based update operator) でoptical flowを推論
  - 図中のLはLookup操作 → 各画素 (u,v) について現在推論したoptical flowによりその移動先がわかるので、移動先の近傍の類似度 (9x9近傍を見るなら81個分の類似度ベクトル) を特徴量とする操作
  - これにより GRU は、『推論中のoptical flow』『そのoptical flowで移動した場合の類似度分布 (全画素分)』『画像1の特徴マップ』からoptical flowを予測する



# まとめ：オプティカルフロー

- 2枚の画像内におけるある物体の移動量をベクトルとして表現したもの
- 動画内の『物体追跡』や『物体の動きの解析』などに利用される
- ブロックマッチング法:
  - 着目点を中心とする窓領域をテンプレートとしてテンプレートマッチング
- Lucas-Kanade法:
  - オプティカルフローの拘束式を立て、隣接画素の動きが似ている (空間的な連続性) と仮定し、複数の拘束式よりオプティカルフローを計算





## 練習問題

$$\begin{cases} 2x + 3y = 7 \\ 4x - y = 1 \end{cases}$$
 この方程式を逆行列を用いて解け

$$\begin{cases} 2x + y = 4 \\ x - y = 0 \\ x + y = 2 \end{cases}$$
 この方程式の近似解を擬似逆行列を用いて解け

(3) 講義で紹介したoptical flow 推定がうまく動作しない可能性が最も高い状況はどれか。最も適切なものを1つ選択せよ。

- A. 入力画像がピンボケしている場合
- B. 輝度が均一な領域が多く存在する被写体が移動する場合
- C. 被写体がカメラの奥行方向に移動する場合
- D. 被写体がカメラに対して縦・横方向に移動する場合

25

Contents : 画像領域分割

## 動的輪郭モデル Active Contours

27

### 動的輪郭モデル (Active Contours)

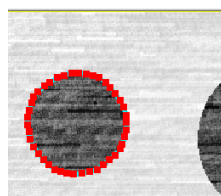
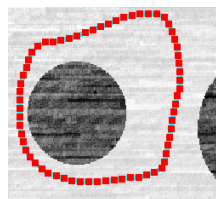
領域境界を徐々に変形する手法

- 境界形状を滑らかに
- 境界が画像のエッジを通るよう

2手法に分類できる

- 境界を陽的に表現する手法 : **Snakes法**
- 境界を陰的に表現する手法 : **Level Set法**

TActiveContour.exe



画像のCTデータは理研・画像情報処理研究チームより

28

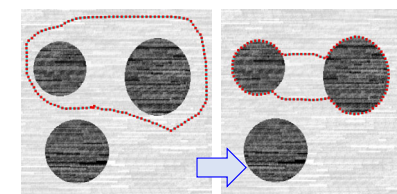
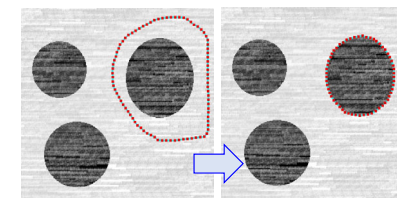
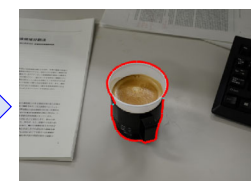
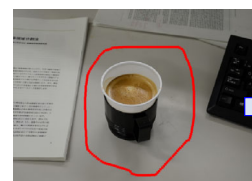
### Snakes法の特徴

#### 利点

- ノイズに強い領域分割が可能
- 高速かつ実装が簡単

#### 欠点

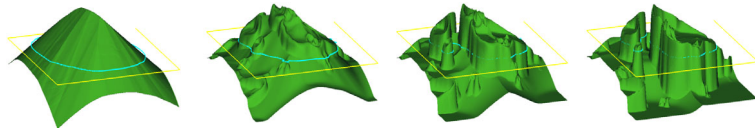
- パラメータに強く依存
- 初期輪郭線に強く依存
- トポロジー変化が困難



29

## Level set法

- 輪郭線を符号付きスカラー場（内挿関数）のゼロセットで表現
- スカラー場の初期値は初期境界からの距離場を利用
- スカラー場を変化させることで輪郭線を間接的に変形する
- → トポロジー変化に対応できる



30

## Level set法の特徴

輪郭線をスカラー場のゼロセットで表現し、スカラー場を変更することで輪郭線を動かす

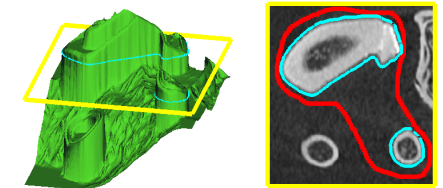
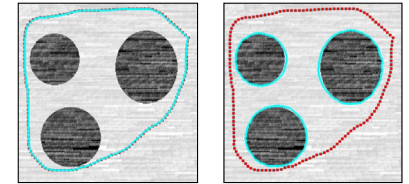
利点

- ノイズに対し堅固・高速

- トポロジー変化に対応

欠点

- 速度 $F(\kappa_{p(t)})$ の適切な選択が難しい
- パラメータ調節が大変



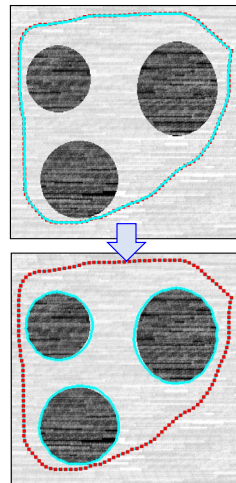
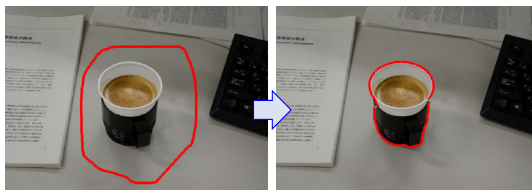
31

## まとめ: 動的輪郭モデル

境界曲線を『形状が滑らかになるように』『画像のエッジを通る様に』変形する手法

- 境界を陽的に表現する: **Snakes** 法
- 境界を陰的に表現する: **Level Set** 法

変形モデル(速度の定義)・最適解の計算法・高次元化など、関連研究は多い



32

概要のみ紹介

Contents: 画像領域分割

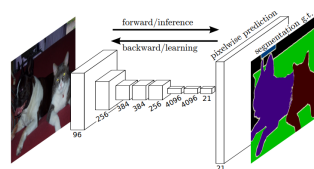
深層学習による領域分割

33

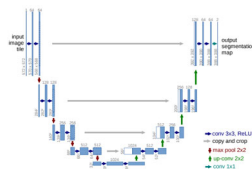


## 深層学習による領域分割

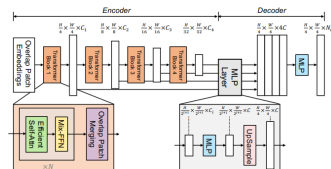
- 画像の各画素にラベルを付与する領域分割に深層学習が利用される
- **学習** : 入力画像とラベル画像のペアを大量に用意  
→ 各入力画像に対してラベル画像を出力できるようにネットワークを最適化
- **推論** : 学習したネットワークに未知画像を入力することでラベル画像を出力  
多様なネットワークモデルが提案される



Fully convolutional Neural Network  
[CVPR2015] 引用数 47547



U-Net  
[MICCAI 2015] 引用数 107945

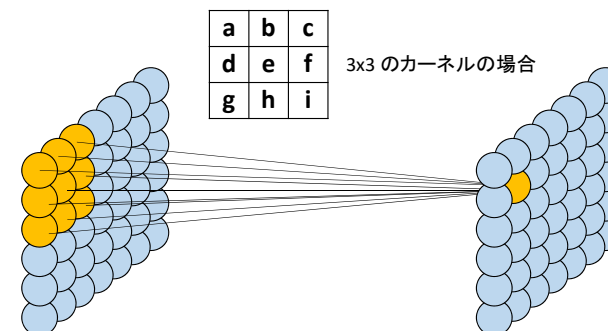


SegFormer  
[NeurIPS 2021] 引用数 5815

34

## Convolutional Neural Network

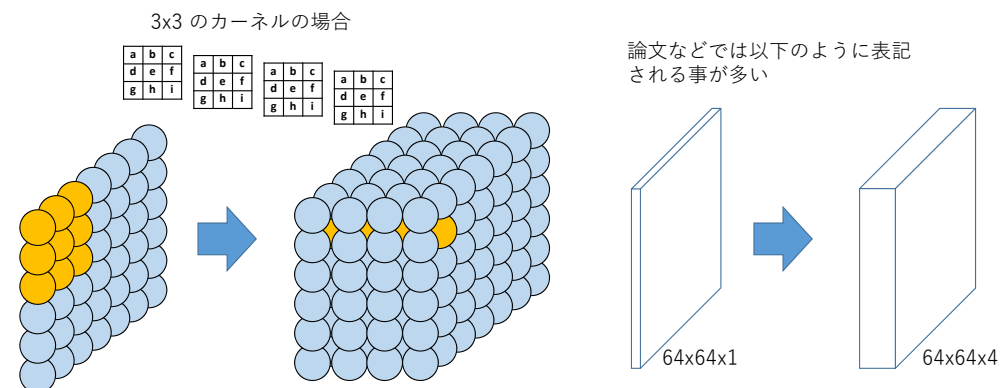
- 主に画像データに対して用いられるNeural Networkの一種
- 畳み込み（線形フィルタ）で入力画像を出力画像に変換
- フィルタの重みを変数で、これを訓練データより調整する



35

## Convolutional Neural Network

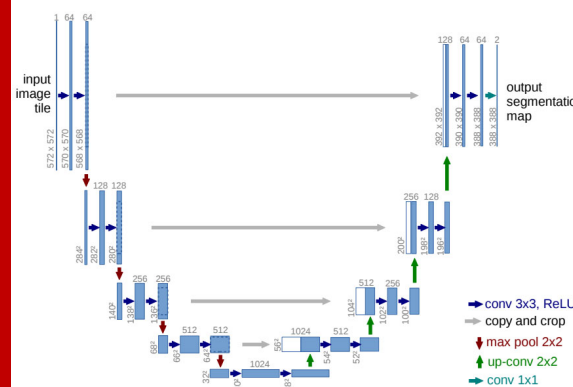
- 線形フィルタを4個用意すると4枚の画像が出力される → channel方向に積み重ねて表現される



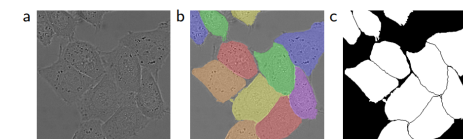
36

## 深層学習による領域分割の例 : UNet

U-Net [MICCAI 2015]



- Encoder Decoder形式 : 入力画像を一度小さなMapに変換し、元のサイズに戻す構造
- 対応する層にスキップ接続があり、元の解像度のmapをそのまま送る
- 細胞画像の分割のために提案され、医用画像等様々な応用がなされる



37

## SAM (Segment Anything Model)

SAM (Segment Anything Model) を動かしてみる  
画面中央の点をシード (プロンプト) としてここからの連結領域の分割が可能

### Step0 仮想環境を作ってactivate (飛ばしてもOK)

フォルダを作成しそこをcurrentにするcommand promptで

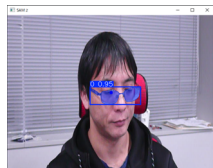
```
> python -m venv cv_sample
> .\cv_sample\Scripts\activate.bat
```

### Step1 必要なライブラリをインストール

```
> python -m pip install opencv-python
> Python -m pip install ultralytics
```

### Step2 右のコードを実行

```
> python samtest.py
```



```
import cv2
from ultralytics import SAM

# SAM 2 をロード
model = SAM("sam2_b.pt")
cap = cv2.VideoCapture(0)

while cap.isOpened():
    success, frame = cap.read()
    if not success:
        break

    # ここがプロンプト
    h, w, _ = frame.shape
    center_point = [w // 2, h // 2]

    results = model.predict(source=frame, points=[center_point], labels=[1], device="cuda")

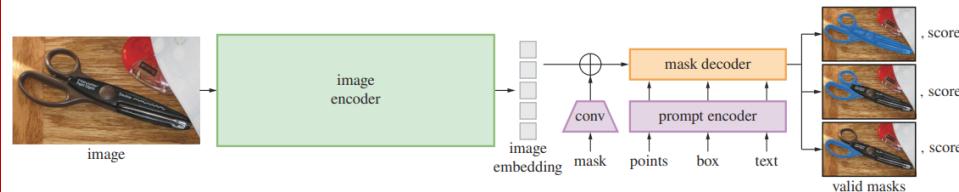
    annotated_frame = results[0].plot()
    cv2.circle(annotated_frame, tuple(center_point), 5, (255, 0, 0), -1)
    cv2.imshow("SAM 2", annotated_frame)

    if cv2.waitKey(1) & 0xFF == ord("q"):
        break

cap.release()
cv2.destroyAllWindows()
```

## SAM (Segment Anything Model)

Segment Anything, ICCV 2023 <https://arxiv.org/abs/2304.02643>



入力: カラー画像 (論文では 1024 x 1024 として実装)

**Image Encoder**: 画像を特徴ベクトルに変換 (64x64ブロックに分割しそれぞれに256次元ベクトル)

- Masked Auto Encoder として訓練したVision Transformer を採用
- MAE → 画像の一部をマスクした状態で入力し元画像を出力させる学習法 (in painting的な感じ)

**Mask Decoder**: 複数のマスクと 確信度を出力

- プロンプト (点・矩形・テキスト) を256次元の特徴ベクトル化したものも受け取る
- 3枚の異なるスケールのマスクと確信度を出力 (全体、中間、部分)

YOLOなどとは違い「なにか」を気にせずマスク (境界) 出力のみを学習

## SAM (Segment Anything Model)

### プロンプトをembedに変換する

点: positional encodingして、前景/背景を示すembedを加算

矩形: 左上の点をpositional encodingして「左上」を示すembedをたす (右下の点も同様にして、ペアを渡すことに)

テキスト: CLIPでencoding

### Light weight mask decoder

**Self attention**: 複数tokenがある場合にtoken同士のattention重みを計算して、その重みでtokenのembedを計算し直す

**Token to image attention**: 各tokenが64x64ブロックのどこに関連するかというattention重みを計算→その重みで64x64ブロックのembedを足して tokenのembedを更新

※ Query = 各トークン、Key & Value: 各ブロック

※ Attentionを通るたびに 位置Encodingを加算 (位置情報喪失防止)

濃いオレンジの矩形を2回通した後...

- 画像の特徴マップを2xの畳込み層に2回通して4倍に (256x256x32)
- 各トークンを「token to image attention」と「MLP」に通して3個の32次元embedにする
- 画像特徴マップの各画素と、トークン由来の特徴ベクトルの内積を取りしきい値処理してマスクに

※最終処理で複数トークンをひとつの特徴ベクトルにまとめ、画像の各画素の特徴ベクトルと内積を取る設計が面白い

### 補足資料

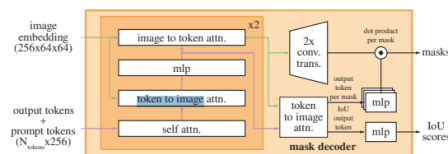


Figure 14: Details of the lightweight mask decoder. A two-layer decoder updates both the image embedding and prompt tokens via cross-attention. Then the image embedding is upsampled, from which the updated output tokens are used to dynamically predict masks. (Not illustrated for figure clarity: At every attention layer, positional encodings are added to the image embedding, and the entire original prompt token (including position encoding) is re-added to the token queries and keys.)

トークン側のAttentionでは、出力時のLinear層にて2048次元に広げてから256に落とす  
一方画像側のAttentionでは、QKVを作る際に128次元に落とし、出力時のLinear層で256に戻す

## まとめ: 深層学習による領域分割

### 現在の領域分割の主流

- 学習**: 入力画像とラベル画像のペアを大量に用意 → 各入力画像に対してラベル画像を出力できるようにネットワークを最適化
- 推論**: 学習したネットワークに未知画像を入力することでラベル画像を出力

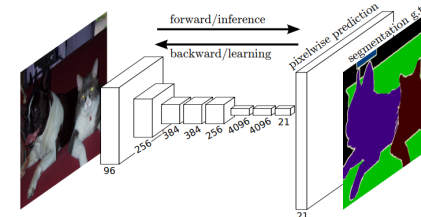
### 利点

- 研究者がアルゴリズムを設計する必要なし!
- 高精度な結果が得られる

### 欠点

- 学習に大量のデータが必要
- 学習に大量の計算資源が必要

※研究テーマとする場合は大規模なモデルをどう使うかに焦点を置くと面白そう



Fully convolutional Neural Network  
[CVPR2015] 引用数 47547

## まとめ：画像領域分割

- ここでは多様な領域分割法を広く浅く紹介した

閾値法 領域成長法 クラスタリング

グラフカット法 動的輪郭モデル 曲面再構成法 深層学習

- 任意の画像（写真, CT, MRI, 顕微鏡）、任意の関心領域(人物、臓器、腫瘍、細胞内小器官)に対し良い結果を出せる『オールマイティ』な領域分割法は未だ実現されていない
- 最近は深層学習が広く利用されるが、訓練データを作るために古典的な手法が必要となることも少なくない

42

## 練習問題

(1) 領域分割に関する次の記述のうち、正しいものをすべて選べ。

- A. 領域分割は、画像を意味的または視覚的に一貫した領域に分ける処理である
- B. 閾値処理は、画素の輝度値や色に基づいて分類する方法の一種である
- C. 領域分割は、入力画像のノイズの影響を受けることはない
- D. クラスタリングに基づく領域分割は、ピクセルの空間的な位置を必ず考慮する
- E. 領域成長法は、seedより領域を成長させる手法である
- F. すべての領域分割手法は、事前に領域数（クラスタ数）を指定する必要がある

(2) 畳込みNNを利用した領域分割モデルが、一度も見たことがない対象画像を正しく領域分割するために特に重要なものを3つ選択せよ

- A. 入力画像のファイル形式を統一すること
- B. 分割対象を十分に考慮して設計された特徴ベクトル抽出
- C. 質や量が十分な教師データ
- D. 畳込みNNのパラメータ量の数十倍程度のメモリ領域
- E. 教師データをある程度再現できる十分な学習
- F. 丁寧に作り込まれたアーキテクチャ構造（NNを組み合わせた計算モデル）
- G. 教師データから学習したハズレ値データに左右されない堅固な特徴空間分割手法

43

## モーフォロジー演算

45

## モーフォロジー演算

集合論の概念を利用した画像変換法

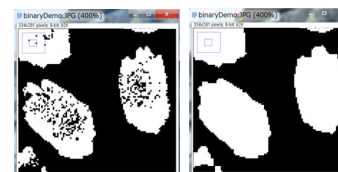
空隙/ノイズ除去・背景グラデーション除去などに利用可能



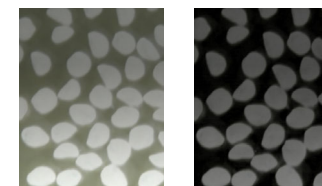
Opening: 細かなごみを除去



Dilation-ErosionでEdge抽出

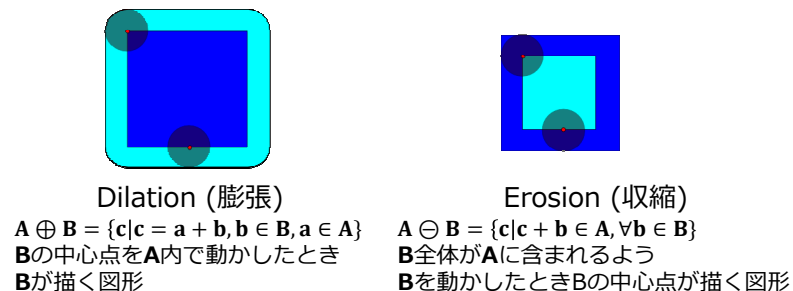
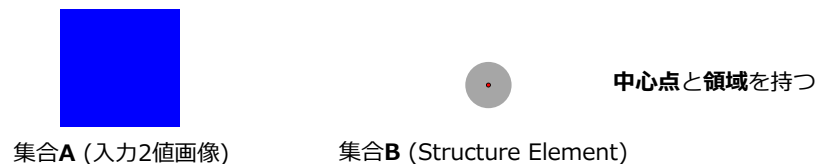


Closing: 領域内の穴を除去



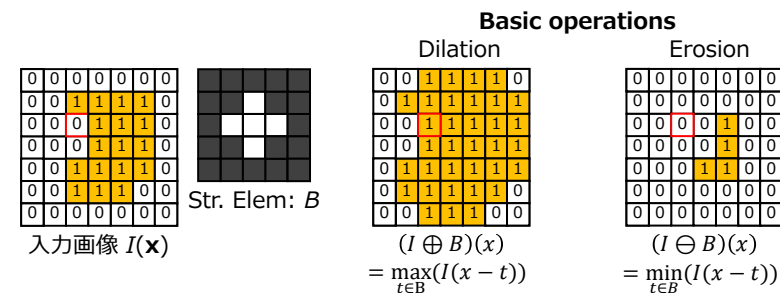
Top-hat: グラデーションを除去

## モーフォロジー演算



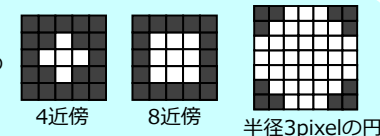
図はwikipediaより [右下の図: CC-BY-SA, BY Renato Keshet]

## 2値画像のMorphological operation (1/3)



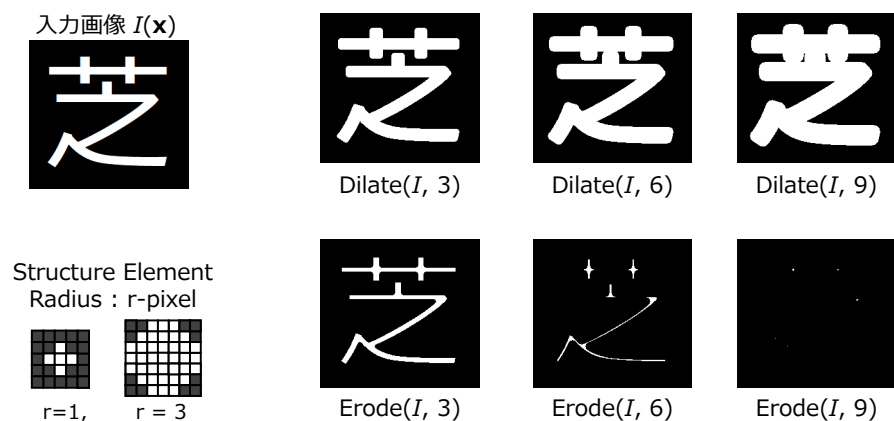
### Structure Element

- + 2値の線形フィルタのようなもの
- + 円形のものが良く用いられる



※細かいことだが、 $\max(I(x-t))$ の『マイナス-』は、(ほとんどないけど)左右/上下非対称なStructure elementを利用するとき大切。計算時は注目画素の周囲の領域の  $\max / \min$  を見るため Structure Elementをひっくり返す必要がある。

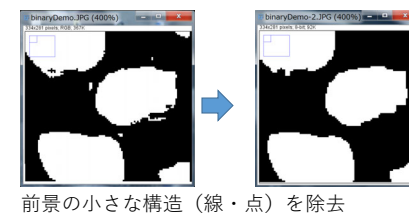
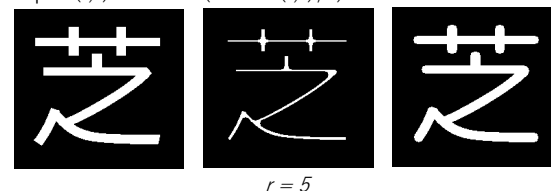
## 2値画像のMorphological operation (2/3)



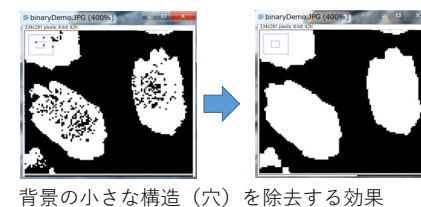
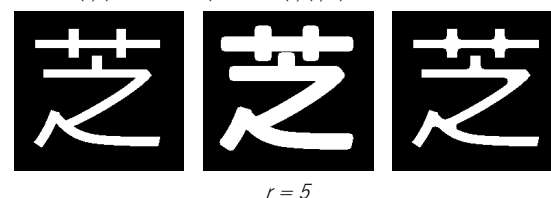
- ※ Dilate(画像, 半径), Erode(画像, 半径),
- ※ Dilateでは、Structure elementが円なので角が取れて膨張する
- ※ Erodeでは、Structure element半径より細い構造はすべて消える

## 2値画像のMorphological operation (3/3)

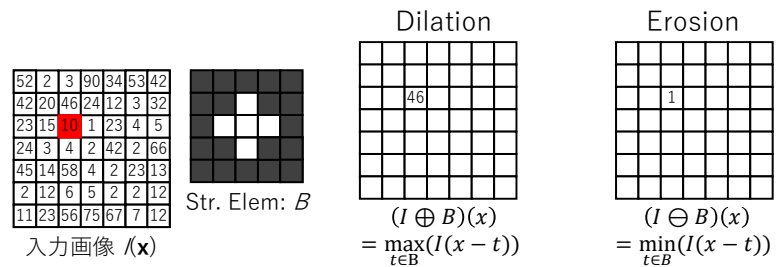
Opening (穴あけ) - 収縮させて → 膨張させる  
 $\text{Open}(I, r) = \text{Dilate}(\text{Erode}(I, r), r)$



Closing (穴うめ) - 膨張させて → 収縮する  
 $\text{Close}(I, r) = \text{Erode}(\text{Dilate}(I, r), r)$

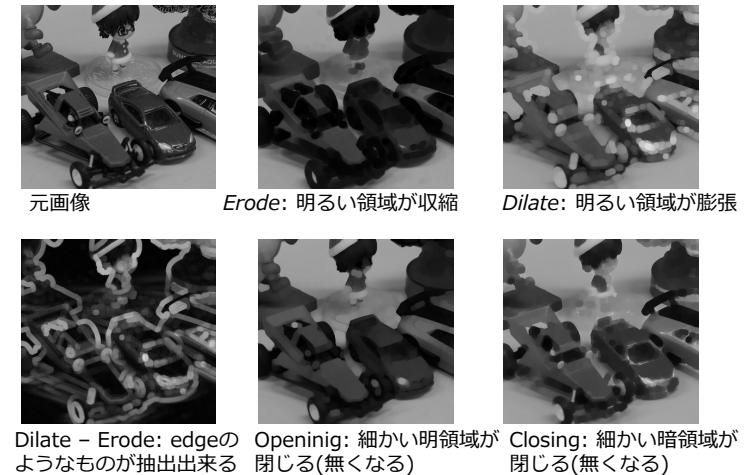


## グレースケール画像のMorphological operation



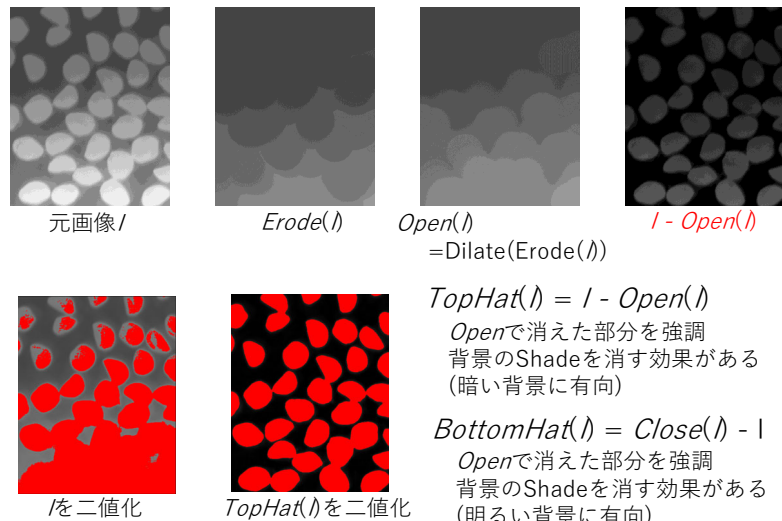
注目画素にStructure Elementを重ね、  
周囲の最大値/最小値を新たな画素値とする

## グレースケール画像のMorphological operation



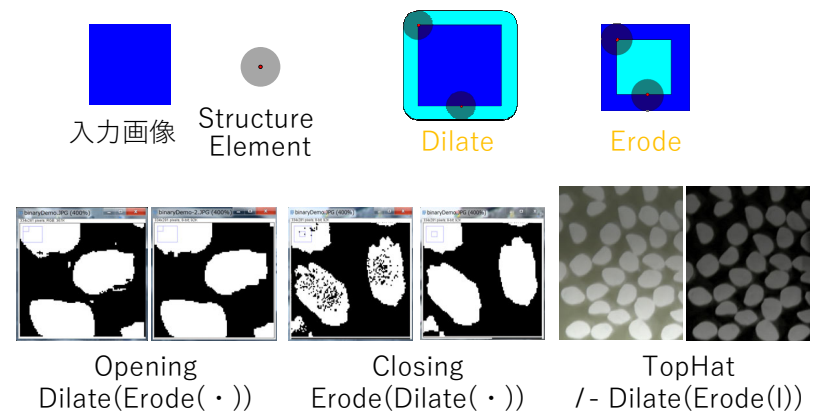
Structure elementは、すべて  $r = 10$  の円

## Top-hat transform による背景除去



## まとめ：モーフォロジー演算

集合理論に基づく画像処理法





Contents : 画像領域分割

## 曲面再構成法

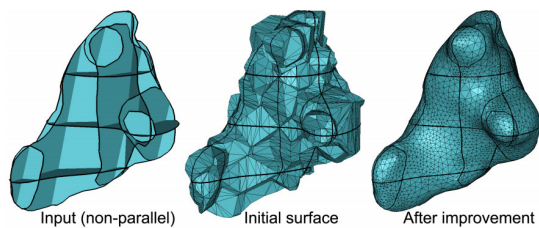
55

56

## 曲面再構成法による領域分割

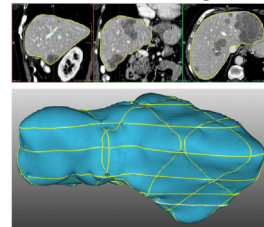
- 形状モデリングのための曲面再構成法を転用
- **輪郭線制約から三次元境界曲面を生成**する

### 陽的曲面再構成法[Lie et. al. 2008]



図は論文[Lie et. al. 2008]より

### 陰関数曲面再構成[Heckel et. al. 2011]

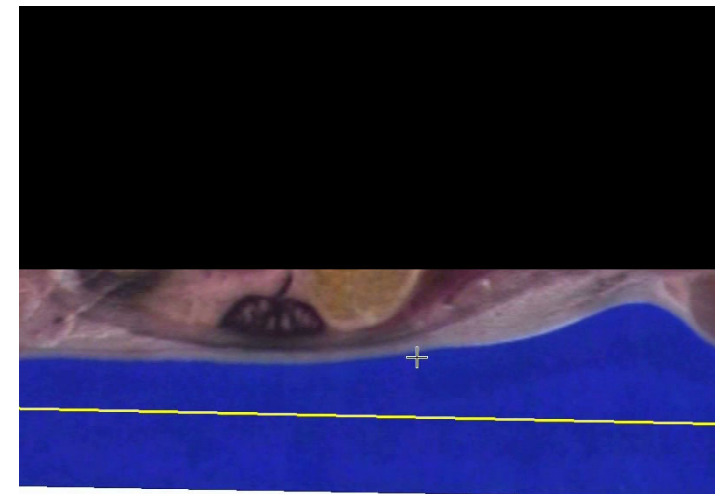


図は論文[Heckel et. al. 2011]より

Heckel F., et. al. : Interactive 3D medical image segmentation with energy-minimizing implicit function. VCBM 35, 2(2011),275-287.  
Liu L. et. al. : Surface reconstruction from non-parallel curve networks. CGF 27, 2(2008), 155-163.

57

## 陰関数曲面再構成による領域分割 [Ijiri et al 2013]



58

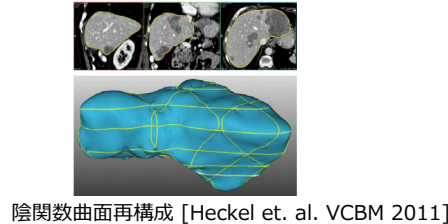
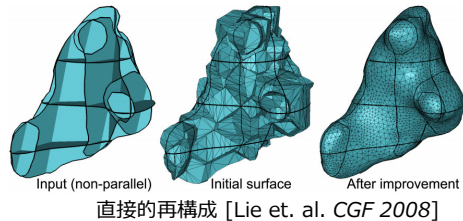


## まとめ：曲面再構成法を応用した領域分割

輪郭線制約から境界面を生成する

**陽的曲面再構成**：輪郭線頂点を直接つなぎ境界面を構築

**陰関数曲面再構成**：輪郭線から滑らかなスカラー場を構築しそのゼロ等値面を出力



TURK G., O'BRIEN J. F.: Modelling with implicit surfaces that interpolate. *ACM TOG* 21, 4(2002), 855-873.  
Heckel F., et. al.: Interactive 3D medical image segmentation with energy-minimizing implicit function. *VCBM* 35, 2(2011), 275-287.  
Liu L. et. al.: Surface reconstruction from non-parallel curve networks. *CGF* 27, 2(2008), 155-163.  
Takashi Ijiri, et. al. Bilateral Hermite Radial Basis Functions for Contour-based Volume Segmentation. *CGF*, 2013.

59

## Snakes法のコスト関数

**前提1.** 曲線はパラメータ表現される  $\mathbf{v}(s) = \begin{pmatrix} x(s) \\ y(s) \end{pmatrix} s \in [0,1]$

**前提2.** 曲線のエネルギー

$$E(\mathbf{v}(s)) = \alpha E_{len} + \beta E_{curv} + \gamma E_{img}$$

弧長に対応する項 (長い → 大)

$$E_{len} = \int_0^1 \left\| \frac{d\mathbf{v}(s)}{ds} \right\|^2 ds$$

曲率に対応する項 (ガタガタ → 大)

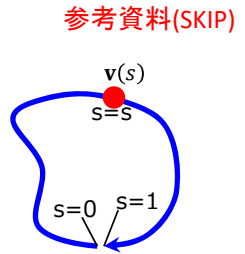
$$E_{curv} = \int_0^1 \left\| \frac{d^2\mathbf{v}(s)}{ds^2} \right\|^2 ds$$

画像(勾配強度)に対応する項

$$E_{img} = - \int_0^1 \| \nabla (G \otimes I(\mathbf{v}(s))) \| ds$$

画像  
ガウスフィルタ(ぼかす)  
勾配強度を計算

**問題**  $E$ を最小化する曲線 $\mathbf{v}(s)$ を探す  
(動的計画法 / 貪欲法など)



60

## Snakes法のコスト関数

最適化計算のため折れ線近似する

$$\mathbf{v}(s) = \begin{pmatrix} x(s) \\ y(s) \end{pmatrix} s \in [0,1]$$

$$E_{len} = \int_0^1 \left\| \frac{d\mathbf{v}(s)}{ds} \right\|^2 ds$$

$$E_{curv} = \int_0^1 \left\| \frac{d^2\mathbf{v}(s)}{ds^2} \right\|^2 ds$$

$$E_{img} = - \int_0^1 \| \nabla (G \otimes I(\mathbf{v}(s))) \| ds$$

$$\begin{aligned} E_{len} &\approx \sum_{i=1}^N (\mathbf{v}_i - \mathbf{v}_{i-1})^2 \\ E_{curv} &\approx \sum_{i=1}^N (\mathbf{v}_{i+1} + \mathbf{v}_{i-1} - 2\mathbf{v}_i)^2 \\ E_{img} &\approx - \sum_{i=1}^N \| \nabla I(\mathbf{v}_i) \| \end{aligned}$$

※左式は連結部分  
 $\mathbf{v}_1$ と $\mathbf{v}_N$ を考慮してない  
ので注意 (見やすく  
するためにこうしました)

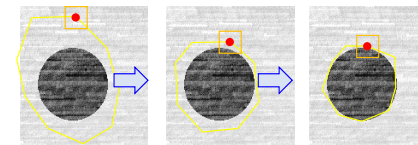
※  $\nabla I$ は勾配強度画像

61

参考資料(SKIP)

## Snakes法の最適化

・色々な最適化法が考えられるがここでは貪欲法を紹介



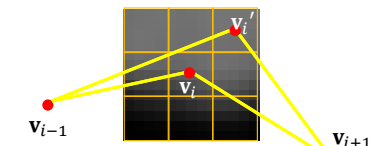
頂点 $\mathbf{v}_i$ が寄与するエネルギー

$$\begin{aligned} E &= \frac{\alpha}{2} (\mathbf{v}_i - \mathbf{v}_{i-1})^2 + \frac{\alpha}{2} (\mathbf{v}_{i+1} - \mathbf{v}_i)^2 \quad (E_{len}) \\ &+ \beta (\mathbf{v}_{i+1} + \mathbf{v}_{i-1} - 2\mathbf{v}_i)^2 \quad (E_{curv}) \\ &- \gamma I'(\mathbf{v}_i) \quad (E_{img}) \end{aligned}$$

### Snakes (貪欲法)

入力：画像と初期曲線 (折れ線)

1. 頂点をひとつずつ訪問する
2. 頂点 $\mathbf{v}_i$ に訪問中、局所コストが最小となる近傍画素へ $\mathbf{v}_i$ を移動する
3. 頂点の移動量が閾値以下になるまで、又は、決められた回数、(1)を繰り返す



- 1-1  $\mathbf{v}_i$ を近傍画素中心に移動してみる
- 1-2 局所エネルギー最小の画素に移動

※この手法だと接線方向に頂点が動いて  
しまう問題があります

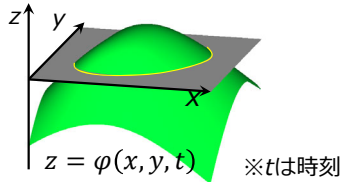
62

参考資料(SKIP)

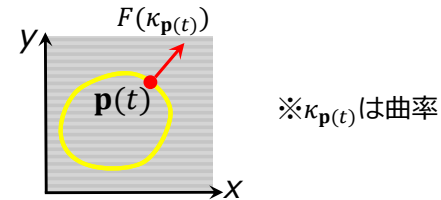
# Level set法

**前提1.** 境界曲線を内挿関数  $\varphi(x, y, t)$  のゼロセットで表現

曲線:  $\varphi(p_x(t), p_y(t), t) = 0$



**前提2.** 曲線上の点  $\mathbf{p}(t)$  は法線方向に速度  $F(\kappa_{\mathbf{p}(t)})$  で動く



**課題:** 収束するまで曲線を变形する

※ 実際に変化させるのは  $\varphi(x, y, t)$

※ 移動速度  $F(\kappa_{\mathbf{p}(t)})$  をうまく指定すると、輪郭の進展が境界で止まる

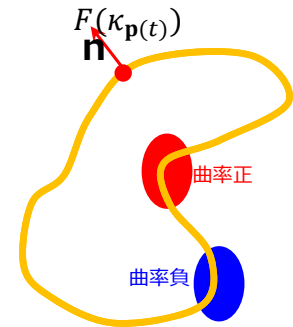
## 速度 $F$ を曲率 $\kappa_{\mathbf{p}(t)}$ の関数にする理由

曲線の凸部・凹部で挙動を変化できる

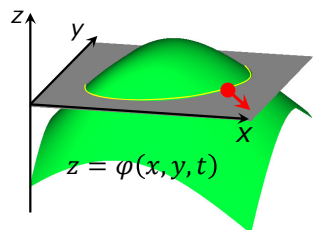
例) 凹部は外へ、凸部は内へ向かって移動

曲がり具合に応じて挙動を変化できる

例) 曲がり具合が大きいところは速く移動



# Level Set法



## 問題の前提

点  $\mathbf{p}$  が曲線に乗る  $\varphi(p_x(t), p_y(t), t) = 0$

点  $\mathbf{p}$  は法線  $\mathbf{n}$  方向に速度  $F$  で移動  $\frac{\partial \mathbf{p}(t)}{\partial t} = F(\kappa_{\mathbf{p}(t)}) \mathbf{n}$

法線  $\mathbf{n}$  は  $\varphi$  の勾配  $\mathbf{n} = \frac{\nabla \varphi}{\|\nabla \varphi\|}$

内挿関数  $\varphi$  の時間進展に関する方程式を導く

整理する  $\frac{\partial \varphi(p_x(t), p_y(t), t)}{\partial t} = -F(\kappa_{\mathbf{p}(t)}) \|\nabla \varphi\|$  ※この式は曲線上で成立  
※変形の詳細は付録へ

全体に拡張  $\frac{\partial \varphi(x, y, t)}{\partial t} = -F(\kappa) \|\nabla \varphi\|$  ※ $\kappa$ は内挿関数  $\varphi$  の曲率

位置  $(x, y)$  と時刻  $t$  について離散化  $\frac{\varphi(i, j, t+1) - \varphi(i, j, t)}{h} = -F(\kappa_{i,j}) \|\nabla \varphi(i, j, t)\|$  ※  $i, j$  は画素位置  
※  $h$  は微小時刻

- 1. 初期曲線から初期スカラー場  $\varphi(i, j, 0)$  を構築

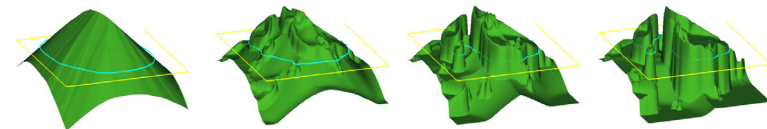
( $i, j$  は画素位置,  $\varphi(i, j, 0)$  は初期曲線からの距離場)

- 2. 変化が十分小さくなるまで時刻  $t$  を進め  $\varphi$  を更新

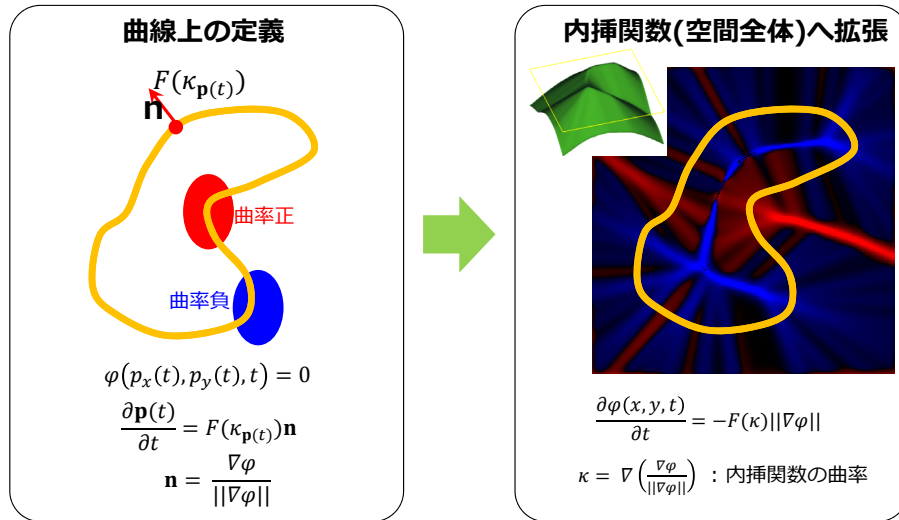
$$\frac{\varphi(i, j, t+1) - \varphi(i, j, t)}{h} = -\frac{a - b\kappa_{i,j}}{1 + \|\nabla(G \otimes I(i, j))\|} \|\nabla \varphi\|$$

$a, b$  はパラメータ  
 $\kappa_{i,j} > a/b$  なら正方向に進む  
エッジ部分では変化が遅い

- 3.  $\varphi(i, j, t) = 0$  である画素を境界として出力する



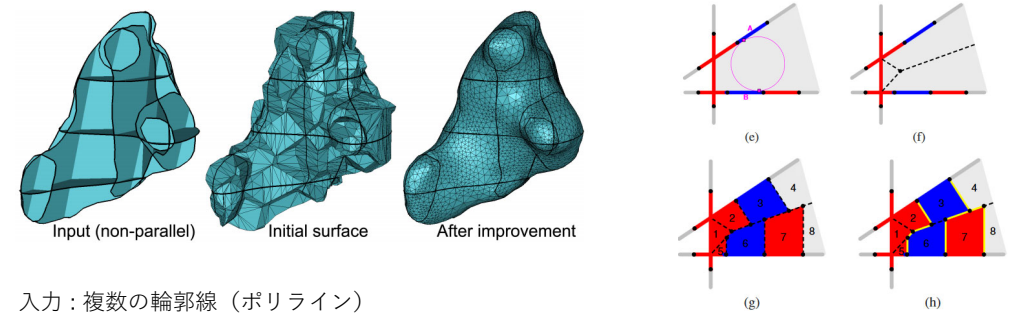
## 曲率の定義



67

## 陽的曲面再構成による領域分割

図は[Liu L. et. al. : Surface reconstruction from non-parallel curve networks. CGF 27, 2(2008), 155-163.]より



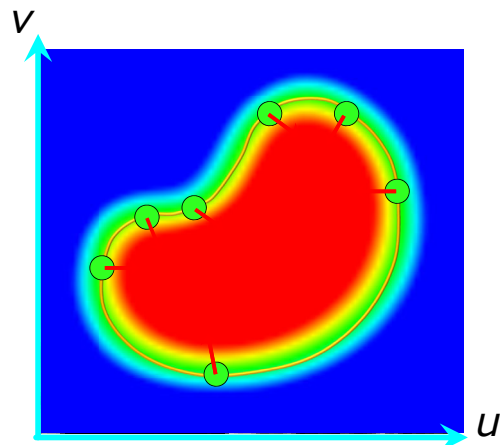
入力：複数の輪郭線（ポリライン）

+ 輪郭線頂点をうまくつなぐことで初期メッシュモデルを生成

+ 初期メッシュモデルを平滑化することで滑らかな境界面を取得する

68

## 曲面再構成法：陰関数曲面再構成 [Turk and O'Brien 02]

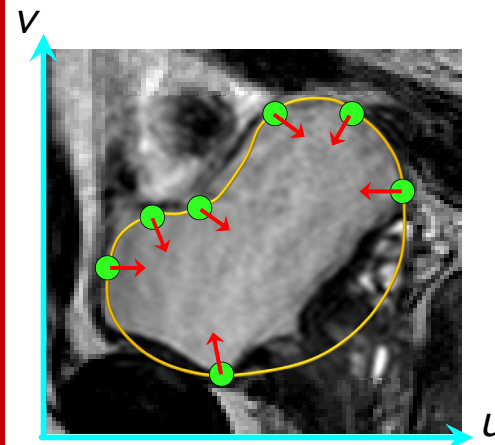


図は[Ijiri et al. EUROGRAPHICS 2013]より

1. 境界の通る点・法線を指定
2. uv空間にスカラー場 $f$ を構築  
 $f(\mathbf{x}) = 0, \nabla f(\mathbf{x}) = \mathbf{n}$
3. スカラー場のゼロセットを抽出

69

## 陰関数曲面再構成による領域分割



図の出典[Ijiri et al. EUROGRAPHICS 2013]

### 問題：画像のエッジを追従しない

→ 動的輪郭モデルで境界を動かす  
[Aliroteh M et. al. 2007]→ Bilateral空間への拡張  
[Ijiri et al 2013]

70

## 陰関数曲面再構成による領域分割 [Ijiri et al 2013]

